

EFDI — Manual Setup Guide (No Docker)

Path 2: Local Development with Hot-Reload

Use this guide when you want to actively develop the project with live code reloading. The backend and frontend run directly on your machine — no containers involved. For a clean deployment on a new machine with no development intent, use the Docker Compose guide instead.

Prerequisites

Install all three of these on the new machine before starting.

Python 3.12

Mac:

```
bash

brew install python@3.12
python3.12 --version    # verify
```

Ubuntu/Debian:

```
bash

sudo apt-get update
sudo apt-get install -y python3.12 python3.12-venv python3.12-dev
python3.12 --version
```

Windows: Download the installer from python.org. During install, check "**Add Python to PATH**".

Why 3.12: The backend (`Dockerfile`) uses (`python:3.12-slim`). Using a different Python version locally risks subtle incompatibilities with pinned packages in (`requirements.txt`).

Node.js 22

Mac:

```
bash

brew install node@22
node --version    # should say v22.x
npm --version
```

Ubuntu/Debian:

```
bash

curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
node --version
```

Windows: Download from nodejs.org — choose the v22 LTS installer.

Why 22: The frontend (`Dockerfile`) uses (`node:22-slim`). The (`package-lock.json`) was generated with npm from Node 22, so using Node 18 or 20 can produce lockfile conflicts on (`npm ci`).

PostgreSQL 16

Mac:

```
bash

brew install postgresql@16
brew services start postgresql@16
```

Ubuntu/Debian:

```
bash

sudo apt-get install -y postgresql-16
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

Windows: Download from [postgresql.org](https://www.postgresql.org). The installer includes pgAdmin and sets up the service automatically.

Why needed locally: Unlike Docker Compose (where Postgres runs in a container), here the backend connects to a Postgres instance running directly on your machine.

Step 1 — Extract the Project

```
bash

unzip efdi.zip
cd efdi
```

Your working directory structure:

```
efdi/
├── backend/
├── frontend/
├── docker-compose.yml    ← not used in this guide
├── .env                  ← docker-compose env, not used here
└── .env.example          ← root-level example, also not used here
```

The root `.env` and `.env.example` are for Docker Compose only. For manual setup, the backend has its own `backend/.env.example` — that is what you use.

Step 2 — Create the Database

Why: The backend connects to Postgres via `DATABASE_URL`. The database, user, and password must exist before running migrations.

```
bash

# Open a Postgres shell as the superuser
# Mac/Linux:
psql -U postgres

# If that errors on Linux, use:
sudo -u postgres psql
```

Inside the Postgres shell, run:

```
sql

CREATE DATABASE efdi;
CREATE USER efdi_user WITH PASSWORD 'efdi123';
GRANT ALL PRIVILEGES ON DATABASE efdi TO efdi_user;

-- PostgreSQL 15+ requires this additional grant:
\c efdi
GRANT ALL ON SCHEMA public TO efdi_user;

\q
```

Why the schema grant: PostgreSQL 15 changed the default privilege model. Without it, Alembic can connect to the database but cannot create tables, and migrations will fail with a `permission denied for schema public` error.

Verify the connection works:

```
bash
```

```
psql -U efdi_user -d efdi -h localhost -c "\conninfo"
```

Step 3 — Configure the Backend Environment

Why: The backend reads all settings from a `.env` file via `pydantic-settings`. Without it, the app refuses to start because `DATABASE_URL` and `SECRET_KEY` are required fields with no defaults.

```
bash  
  
cd efdi/backend  
cp .env.example .env
```

Now open `backend/.env` in your editor and fill in the values:

```
env
```

```
# --- Application ---
APP_NAME="Enterprise Financial Document Intelligence"
APP_ENV=development          # keeps seed_users.py working; disables strict SECRET_KEY
APP_DEBUG=true
API_V1_PREFIX=/api/v1

# --- Database ---
DATABASE_URL=postgresql+psycopg2://efdi_user:efdi123@localhost:5432/efdi

# --- Security / JWT ---
SECRET_KEY=any-random-string-for-local-dev-only  # generate with: openssl rand
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=7

# --- File Storage ---
UPLOAD_DIR=uploads
MAX_UPLOAD_SIZE_MB=25

# --- Bulk Intake ---
INTAKE_SCAN_DIR=intake
MAX_BULK_INTAKE_FILES=100

# --- Logging ---
LOG_LEVEL=INFO
LOG_DIR=logs

# --- OCR ---
# Start with stub to confirm everything works before adding OCR
OCR_DEFAULT_ENGINE=stub

# --- CORS ---
# Vite dev server runs on 5173 - both origins needed if you test from either
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Key things to understand:

Variable	Why this value
<code>DATABASE_URL</code>	Uses <code>psycopg2</code> (sync driver) — matches what SQLAlchemy and Alembic use in this project
<code>APP_ENV=development</code>	Required for <code>seed_users.py</code> to run (it refuses if <code>APP_ENV=production</code>). Also relaxes the SECRET_KEY 32-char minimum
<code>OCR_DEFAULT_ENGINE=stub</code>	Safe starting point — confirms the full pipeline works before adding OCR model weights
<code>CORS_ORIGINS</code>	Vite proxies <code>/api/*</code> to the backend, but the backend still validates the <code>Origin</code> header. Both <code>localhost</code> and <code>127.0.0.1</code> are listed because browsers treat them as different origins

Step 4 — Set Up the Python Virtual Environment

Why: A virtual environment isolates the project's Python packages from your system Python and from any other projects on the machine. Without it, package versions conflict across projects.

```
bash

# Make sure you are in efdi/backend/
cd efdi/backend

# Create the venv with Python 3.12 explicitly
python3.12 -m venv venv

# Activate it
# Mac/Linux:
source venv/bin/activate
# Windows:
venv\Scripts\activate

# Your prompt should now show (venv)
```

Step 5 — Install Python Dependencies

Core packages (always required)

```
bash

pip install --upgrade pip
pip install -r requirements.txt
```

What this installs: FastAPI, Uvicorn, Gunicorn, SQLAlchemy, Alembic, psycopg2-binary, Pydantic, python-jose (JWT), bcrypt, PyMuPDF, OpenCV, Pillow, scikit-learn, and all pinned transitive dependencies.

OCR packages (required for real OCR, skip for stub)

If you set `OCR_DEFAULT_ENGINE=stub` in Step 3, skip this section for now. Come back once the app is running correctly.

For EasyOCR:

```
bash

# Step A — install OCR packages (easyocr, paddleocr, paddlepaddle)
pip install -r requirements-ocr.txt

# Step B — install PyTorch CPU-only (MUST use PyTorch's own index, not PyPI)
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

Why two steps for torch? The default PyPI `torch` wheel dynamically links against NVIDIA CUDA libraries (`libcudart.so`, `libcublasLt.so`). On a machine without an NVIDIA GPU and drivers, `import torch` crashes at load time — not at runtime — even if you pass `gpu=False`. The CPU-only wheel from `download.pytorch.org/whl/cpu` (~190 MB) has no CUDA dependency and imports cleanly on any machine. EasyOCR requires torch, so this must be installed before EasyOCR will work. This is documented in `backend/requirements-ocr-torch-notes.txt`.

For PaddleOCR:

```
bash

pip install -r requirements-ocr.txt
# PaddleOCR uses paddlepaddle (not torch) so no special index needed
```

Then update `backend/.env`:

```
env
```

```
OCR_DEFAULT_ENGINE=easyocr # or paddleocr
```

Also update `backend/app/ocr/factory.py` if switching to PaddleOCR — see the [PaddleOCR switching guide](#).

Step 6 — Run Database Migrations

Why: Alembic migrations create the full database schema — all 9 tables (users, documents, ocr_results, classification_results, extraction_results, validation_results, audit_logs, workflow_history, training_examples). Without this step, the backend starts but every API call that touches the DB will throw a `relation does not exist` error.

```
bash

# Make sure venv is active and you are in efdi/backend/
alembic upgrade head
```

Expected output (all 9 migrations applied in order):

```
INFO [alembic.runtime.migration] Running upgrade -> 4cf4fda96099 (create
users table)
INFO [alembic.runtime.migration] Running upgrade 4cf4fda96099 ->
2ffaeae37b7d (expand user model)
INFO [alembic.runtime.migration] Running upgrade 2ffaeae37b7d ->
25865efaaa0b (add documents table)
INFO [alembic.runtime.migration] Running upgrade 25865efaaa0b ->
46222a8b1535 (add ocr_results table)
INFO [alembic.runtime.migration] Running upgrade 46222a8b1535 ->
be1d0849d262 (add classification_results)
INFO [alembic.runtime.migration] Running upgrade be1d0849d262 ->
87d0535e0a9c (add extraction_results)
INFO [alembic.runtime.migration] Running upgrade 87d0535e0a9c ->
5cf24992553b (add validation_results)
INFO [alembic.runtime.migration] Running upgrade 5cf24992553b ->
206f2a9dffa6 (add workflow_history)
INFO [alembic.runtime.migration] Running upgrade 206f2a9dffa6 ->
bfd7768c6be6 (add audit_logs)
INFO [alembic.runtime.migration] Running upgrade bfd7768c6be6 ->
b9c2ae51cb79 (add training_examples)
INFO [alembic.runtime.migration] Done.
```

If you see an error like `connection refused` — Postgres is not running. Start it and retry. If you see `permission denied for schema public` — re-run the schema grant from Step 2.

Step 7 — Seed Default Users

Why: The database starts completely empty. The seed script creates one account for each role so you can log in immediately without manually calling the registration API.

```
bash

# Make sure venv is active and you are in efdi/backend/
python scripts/seed_users.py
```

Output:

```
Created 4 user(s): ['admin', 'manager', 'analyst', 'auditor']
Skipped 0 existing user(s): []

Default credentials for local testing:
ROLE            USERNAME      PASSWORD
ADMIN           admin         Admin@123
FINANCE_MANAGER manager       Manager@123
FINANCE_ANALYST analyst       Analyst@123
AUDITOR         auditor       Auditor@123
```

The script is **idempotent** — running it again skips any user that already exists, so it is safe to re-run.

These credentials are for local development only. The script checks `APP_ENV` and refuses to run if it is set to `production`.

Step 8 — Start the Backend

```
bash

# Make sure venv is active and you are in efdi/backend/
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Why `--reload`: Uvicorn watches the `app/` directory and automatically restarts the server when you save a Python file. This is the hot-reload behaviour that makes manual setup useful for development.

Why not Gunicorn here: Gunicorn's multi-worker mode doesn't support hot-reload. Use Uvicorn directly for development.

Verify the backend is running:

```
bash
```

```
curl http://localhost:8000/health
# Expected: {"status": "ok", ...}
```

The interactive API docs are at:

- Swagger UI: <http://localhost:8000/api/docs>
 - ReDoc: <http://localhost:8000/api/redoc>
-

Step 9 — Set Up and Start the Frontend

Open a **second terminal** (keep the backend running in the first one).

```
bash

cd efdi/frontend
npm install
npm run dev
```

Why `npm install` and not `npm ci`: `npm ci` is for CI environments — it requires an exact lockfile match and deletes `node_modules` first. `npm install` is the standard dev command; it installs from `package-lock.json` and is forgiving of minor environment differences.

Vite will print:

```
VITE v8.x ready in Xms

→ Local:   http://localhost:5173/
→ Network: http://YOUR_IP:5173/
```

Open <http://localhost:5173> in your browser.

Why no Nginx needed here: `vite.config.ts` has a built-in proxy rule:

```
/api/* → http://localhost:8000/api/*
```

The browser talks to Vite on port 5173, and Vite forwards any `/api` call to the backend on port 8000. No Nginx, no CORS issues in dev mode.

Step 10 — Log In

Go to <http://localhost:5173> and log in with any of the seeded accounts:

Development Workflow

Making backend changes

Edit any file in `(efdi/backend/app/)`. Uvicorn detects the change and restarts automatically (you will see `(Reloading...)` in the backend terminal).

Making frontend changes

Edit any file in `(efdi/frontend/src/)`. Vite's HMR (Hot Module Replacement) updates the browser instantly without a full page reload.

Adding a new database column or table

```
bash

# After editing the SQLAlchemy model in app/models/
alembic revision --autogenerate -m "describe_your_change"
alembic upgrade head
```

Running backend tests

```
bash

cd efdi/backend
pip install -r requirements-dev.txt # installs pytest and httpx (test-only de
pytest
```

Stopping everything

- Backend: `(Ctrl+C)` in the backend terminal
 - Frontend: `(Ctrl+C)` in the frontend terminal
-

Troubleshooting

`psycpg2` install fails on Mac (Apple Silicon)

The binary wheel (`psycpg2-binary`) in `requirements.txt` handles ARM64 — it should install without issues. If it fails:

```
bash  
  
brew install libpq  
pip install psycpg2-binary --no-cache-dir
```

`alembic upgrade head` fails with `relation "alembic_version" does not exist`

This is normal on the very first run — Alembic creates that table itself. If the error is something else, the most common cause is a wrong `DATABASE_URL` in `backend/.env`. Double-check the username, password, and database name match what you created in Step 2.

Backend starts but login returns 401 / JWT errors

Check that `SECRET_KEY` in `backend/.env` is not the literal placeholder value `change-me-to-a-long-random-string`. The app validates this at startup and will log a clear error if it is. Generate a real one:

```
bash  
  
openssl rand -hex 32
```

Frontend shows a blank page or crashes on load

```
bash  
  
cd efdi/frontend  
npm run build # run a production build to see TypeScript/compile errors clear
```

Check the browser console (F12) for specific error messages.

EasyOCR crashes on first OCR request

Most likely `torch` is not installed or the CUDA-linked PyPI wheel was installed instead of the CPU wheel. Verify:

```
bash  
  
python -c "import torch; print(torch.__version__)"
```

If it errors, reinstall:

bash

```
pip uninstall torch -y
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

Port 8000 or 5173 already in use

bash

```
# Find and kill the process using port 8000
lsof -ti:8000 | xargs kill -9      # Mac/Linux
# For 5173:
lsof -ti:5173 | xargs kill -9
```

Quick Reference

Terminal	Directory	Command
Terminal 1 (backend)	efdi/backend	source venv/bin/activate && uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
Terminal 2 (frontend)	efdi/frontend	npm run dev

URL	What it is
http://localhost:5173	Frontend (React app)
http://localhost:8000/api/docs	Backend Swagger UI
http://localhost:8000/api/redoc	Backend ReDoc
http://localhost:8000/health	Backend health check